Reproduce HOL blocking with default scheduler

argowf yaml: @spark-pi-workflow.yaml

1. simulate a large long-running spark job which consumes all(almost) resources:

```
argo submit spark-pi-workflow.yaml \
    -p image=apache/spark:3.5.7 \
    -p partitions=1000000 \
    -p namespace=yunikorn-spark-submit \
    -p executors=16
```

screenshot:

```
● ➜  spark-submit git:(main) ✗ argo submit spark-pi-workflow.yaml \
      -p image=apache/spark:3.5.7 \
      -p partitions=1000000 \
      -p namespace=yunikorn-spark-submit \
      -p executors=16
  Name:                spark-pi-wf-vg2lb
  Namespace:           yunikorn-spark-submit
  ServiceAccount:      unset (will run with the default ServiceAccount)
  Status:              Pending
  Created:             Sun Jan 04 17:02:02 +0800 (now)
  Progress:
  Parameters:
    image:             apache/spark:3.5.7
    partitions:        1000000
    namespace:         yunikorn-spark-submit
    executors:         16
    queue:             root.default
    scheduler:         default-scheduler
● ➜  spark-submit git:(main) ✗ kgp
  NAME                                  READY   STATUS    RESTARTS   AGE
  pythonpi-c0b3049b883e1f04-exec-1      1/1     Running   0          85s
  pythonpi-c0b3049b883e1f04-exec-10     1/1     Running   0          84s
  pythonpi-c0b3049b883e1f04-exec-11     1/1     Running   0          84s
  pythonpi-c0b3049b883e1f04-exec-12     1/1     Running   0          83s
  pythonpi-c0b3049b883e1f04-exec-13     0/1     Pending   0          83s
  pythonpi-c0b3049b883e1f04-exec-14     0/1     Pending   0          83s
  pythonpi-c0b3049b883e1f04-exec-15     0/1     Pending   0          83s
  pythonpi-c0b3049b883e1f04-exec-16     0/1     Pending   0          82s
  pythonpi-c0b3049b883e1f04-exec-2      1/1     Running   0          85s
  pythonpi-c0b3049b883e1f04-exec-3      1/1     Running   0          85s
  pythonpi-c0b3049b883e1f04-exec-4      1/1     Running   0          85s
  pythonpi-c0b3049b883e1f04-exec-5      1/1     Running   0          85s
  pythonpi-c0b3049b883e1f04-exec-6      1/1     Running   0          85s
  pythonpi-c0b3049b883e1f04-exec-7      1/1     Running   0          85s
  pythonpi-c0b3049b883e1f04-exec-8      1/1     Running   0          84s
  pythonpi-c0b3049b883e1f04-exec-9      1/1     Running   0          84s
  spark-pi-930fdc9b883e03f9-driver      1/1     Running   0          96s
  spark-pi-wf-vg2lb                     2/2     Running   0          98s
● ➜  spark-submit git:(main) argo submit spark-pi-workflow.yaml \
```

2. simulate a small short spark job, which we expect low latency:

```
-p image=apache/spark:3.5.7 \
-p partitions=10000 \
-p namespace=yunikorn-spark-submit \
-p executors=1
```

screenshot:

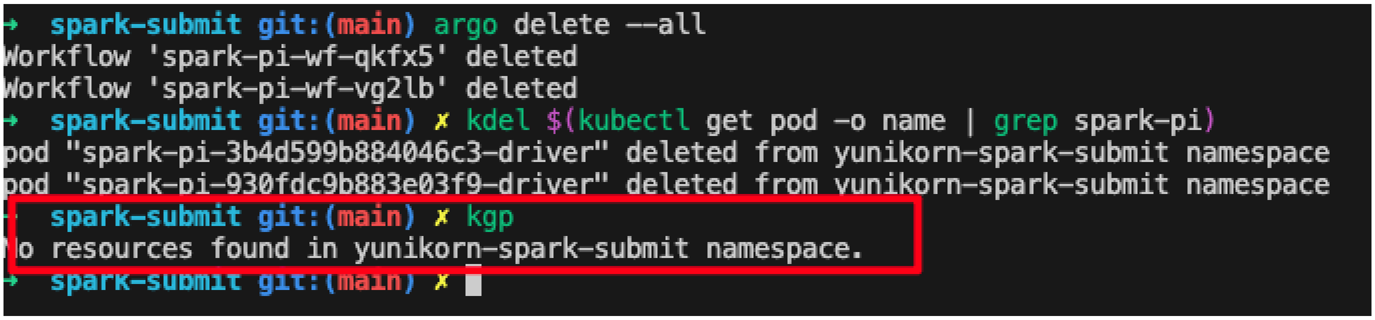
```
● → spark-submit git:(main) argo submit spark-pi-workflow.yaml \
     -p image=apache/spark:3.5.7 \
     -p partitions=10000 \
     -p namespace=yunikorn-spark-submit \
     -p executors=1
Name:                spark-pi-wf-qkfx5
Namespace:           yunikorn-spark-submit
ServiceAccount:      unset (will run with the default ServiceAccount)
Status:              Pending
Created:             Sun Jan 04 17:03:54 +0800 (now)
Progress:
Parameters:
  image:             apache/spark:3.5.7
  partitions:        10000
  namespace:         yunikorn-spark-submit
  executors:         1
  queue:             root.default
  scheduler:         default-scheduler
● → spark-submit git:(main) kubectl get pods --sort-by=.metadata.creationTimestamp
NAME                                  READY   STATUS    RESTARTS   AGE
spark-pi-wf-vg2lb                     2/2     Running   0          4m47s
spark-pi-930fdc9b883e03f9-driver      1/1     Running   0          4m45s
pythonpi-c0b3049b883e1f04-exec-3      1/1     Running   0          4m34s
pythonpi-c0b3049b883e1f04-exec-1      1/1     Running   0          4m34s
pythonpi-c0b3049b883e1f04-exec-7      1/1     Running   0          4m34s
pythonpi-c0b3049b883e1f04-exec-6      1/1     Running   0          4m34s
pythonpi-c0b3049b883e1f04-exec-5      1/1     Running   0          4m34s
pythonpi-c0b3049b883e1f04-exec-4      1/1     Running   0          4m34s
pythonpi-c0b3049b883e1f04-exec-2      1/1     Running   0          4m34s
pythonpi-c0b3049b883e1f04-exec-9      1/1     Running   0          4m33s
pythonpi-c0b3049b883e1f04-exec-8      1/1     Running   0          4m33s
pythonpi-c0b3049b883e1f04-exec-11     1/1     Running   0          4m33s
pythonpi-c0b3049b883e1f04-exec-10     1/1     Running   0          4m33s
pythonpi-c0b3049b883e1f04-exec-15     0/1     Pending   0          4m32s
pythonpi-c0b3049b883e1f04-exec-14     0/1     Pending   0          4m32s
pythonpi-c0b3049b883e1f04-exec-13     0/1     Pending   0          4m32s
pythonpi-c0b3049b883e1f04-exec-12     1/1     Running   0          4m32s
pythonpi-c0b3049b883e1f04-exec-16     0/1     Pending   0          4m31s
spark-pi-wf-qkfx5                     2/2     Running   0          2m55s
spark-pi-3b4d599b884046c3-driver      0/1     Pending   0          108s
○ → spark-submit git:(main)
```

conclusion:

with default scheduler and limited resources, there will be HOL blocking, large long-running spark jobs consume all resources will block future jobs in the same the same ns.

Resolve HOL blocking with yunikorn

1. clear all workloads:



2. install yunikorn and config queues

set up 3 queues:

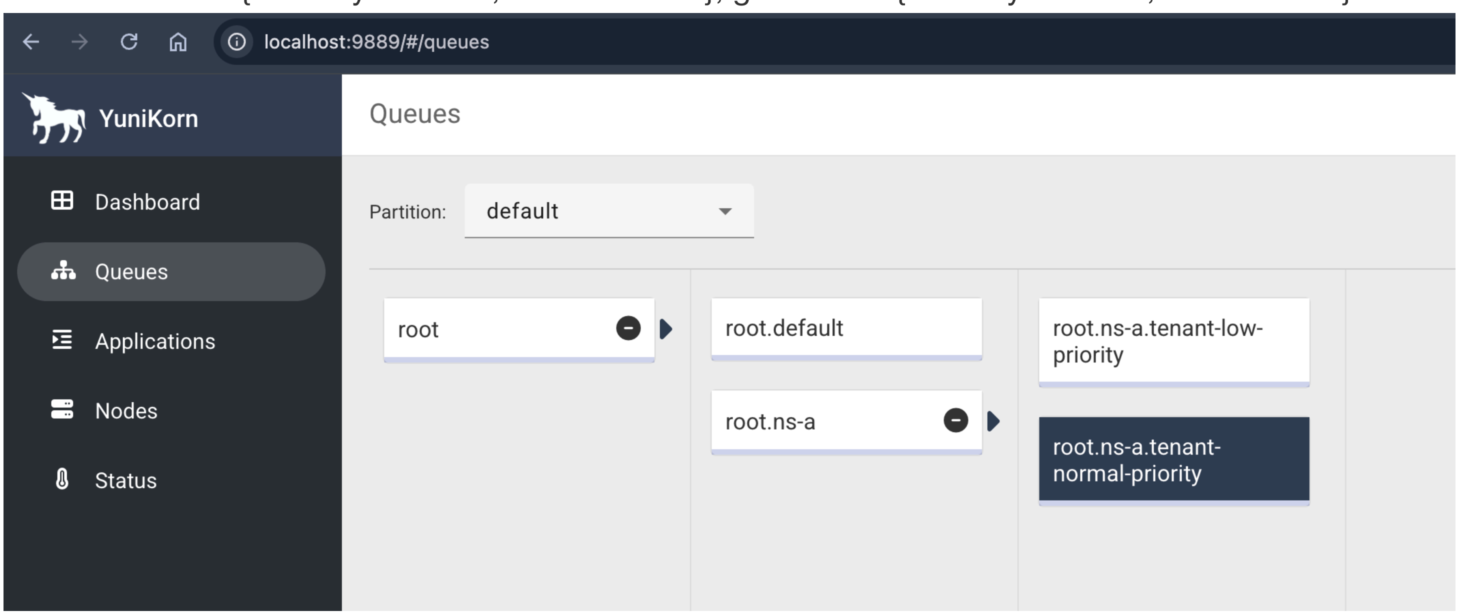
root.default

root.ns-a.tenant-low-priority

-- resource: max{memory: 6000Mi, vcore: 1200m}, guaranteed{memory: 3000Mi, vcore: 600m}

root.ns-a.tenant-normal-priority

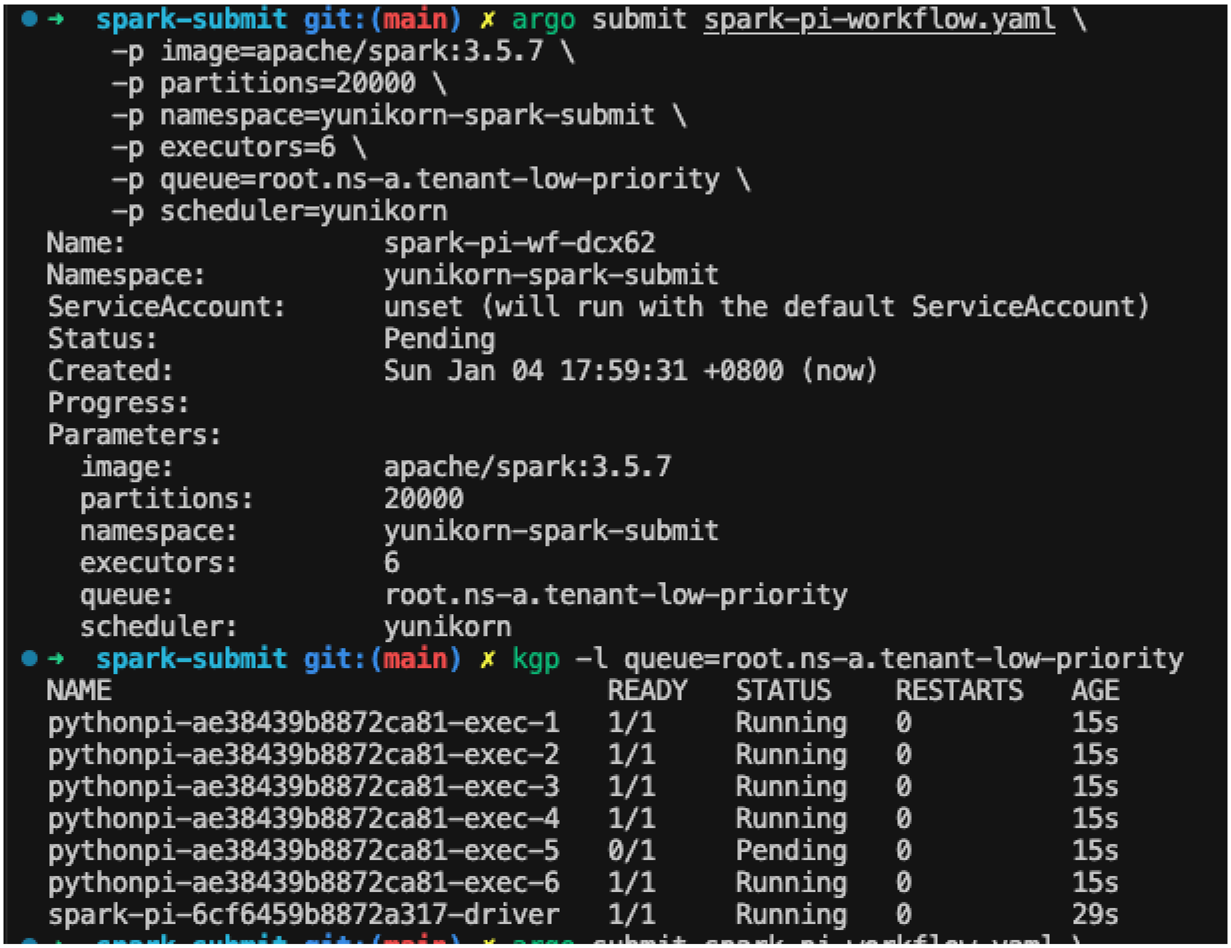
-- resource: max{memory: 6000Mi, vcore: 1200m}, guaranteed{memory: 3000Mi, vcore: 600m}



3. simulate a large long-running spark job which consume all(almost) allocated resources:

specify yunikorn as the scheduler and the queue with low priority

```
argo submit spark-pi-workflow.yaml \
    -p image=apache/spark:3.5.7 \
    -p partitions=20000 \
    -p namespace=yunikorn-spark-submit \
    -p executors=6 \
    -p queue=root.ns-a.tenant-low-priority \
    -p scheduler=yunikorn
```



```
● ➜  spark-submit git:(main) ✗ argo submit spark-pi-workflow.yaml \
      -p image=apache/spark:3.5.7 \
      -p partitions=20000 \
      -p namespace=yunikorn-spark-submit \
      -p executors=6 \
      -p queue=root.ns-a.tenant-low-priority \
      -p scheduler=yunikorn
  Name:                spark-pi-wf-dcx62
  Namespace:           yunikorn-spark-submit
  ServiceAccount:      unset (will run with the default ServiceAccount)
  Status:              Pending
  Created:             Sun Jan 04 17:59:31 +0800 (now)
  Progress:
  Parameters:
    image:             apache/spark:3.5.7
    partitions:        20000
    namespace:         yunikorn-spark-submit
    executors:         6
    queue:             root.ns-a.tenant-low-priority
    scheduler:         yunikorn
● ➜  spark-submit git:(main) ✗ kgp -l queue=root.ns-a.tenant-low-priority
  NAME                               READY   STATUS    RESTARTS   AGE
  pythonpi-ae38439b8872ca81-exec-1   1/1     Running   0          15s
  pythonpi-ae38439b8872ca81-exec-2   1/1     Running   0          15s
  pythonpi-ae38439b8872ca81-exec-3   1/1     Running   0          15s
  pythonpi-ae38439b8872ca81-exec-4   1/1     Running   0          15s
  pythonpi-ae38439b8872ca81-exec-5   0/1     Pending   0          15s
  pythonpi-ae38439b8872ca81-exec-6   1/1     Running   0          15s
  spark-pi-6cf6459b8872a317-driver   1/1     Running   0          29s
```

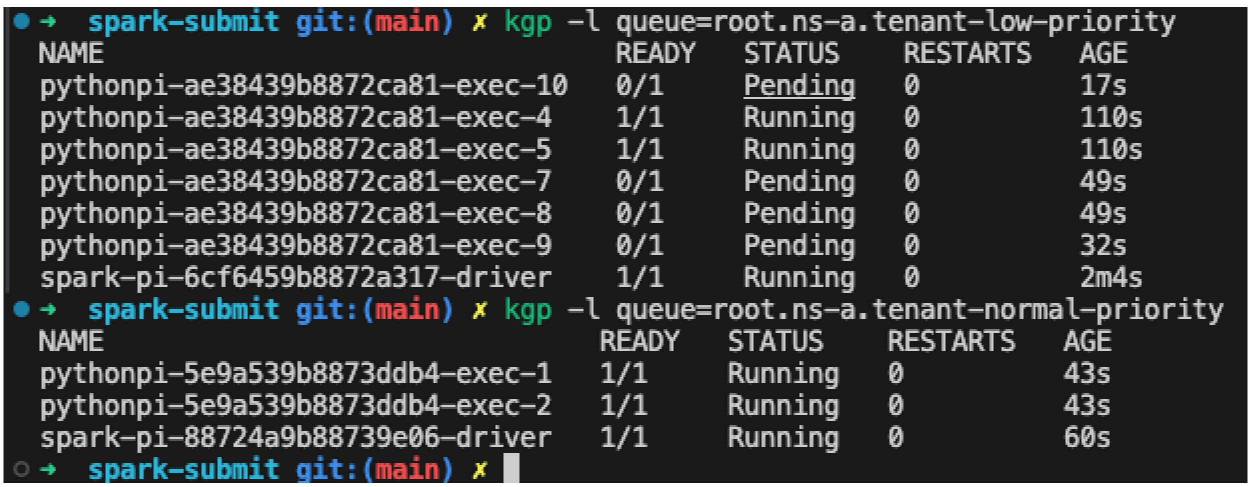
4. simulate a small short spark job, which we expect low latency:

specify yunikorn as the scheduler and the queue with normal priority, which is higher than the previous low priority queue, so can preempt pods from that queue

```
argo submit spark-pi-workflow.yaml \
    -p image=apache/spark:3.5.7 \
    -p partitions=10000 \
    -p namespace=yunikorn-spark-submit \
    -p executors=2 \
    -p queue=root.ns-a.tenant-normal-priority \
    -p scheduler=yunikorn
```

wait for some momemt will see:

1. sompe of the exec pods of the low priority job was preempted, and new ones created but in pending state.
2. the normal priority spark job was able to run, because it runs on a different queue with higher priority, and can preempt pods on the low priority queue when it consumes resources less than its guaranteed.



conclusion:

with yunikorn scheduler and prioritized queues, HOL blocking can be resolved, large long-running spark jobs running on low priority queues can be preempted by jobs running on higher priority queues. thus jobs with higher priority can get resource to execute.